PostgreSQL Extension in C

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual | *Languages: C only*

1 Problem Statement

You will write a PostgreSQL extension in C that adds a new data type, operators on that type, utility functions, and an aggregate function to a running PostgreSQL server. The new type is *semantic version*, the dotted-integer format you have already seen in package managers like npm, pip, cargo, and apt: `1.2.3`, `2.0.0`, `0.14.7`. After your extension is installed, a user of `psql` can declare columns of type `semver`, insert version strings, sort them correctly, query them with operators like `>=`, and use built-in aggregate functions like `MAX` and `MIN` on them. To PostgreSQL and its users, your type is indistinguishable from a built-in.

This is an unusual project because the code you write does not run as a standalone process. It is compiled as a shared library and loaded by the PostgreSQL server, then invoked by PostgreSQL whenever a query touches your type. Your functions execute inside the database backend process, with access to PostgreSQL's internal memory allocator and value representation. This is how production extensions like PostGIS (which adds geographic types), pgvector (which adds vector similarity search), and `pg_stat_statements` (which tracks query performance) work. You are building a small example of the same thing.

The technical content of the project is twofold. First, you learn the PostgreSQL extension mechanism: the PGXS build system, the conventions for declaring functions in C and registering them in SQL, the rules for memory allocation inside the server, and the special macros used to convert between PostgreSQL's universal value representation (`Datum`) and ordinary C values. Second, you learn how a database earns the right to be "extensible" by exposing a small, well-defined C API that lets third-party code add features without modifying the server itself. Once you understand this, the difference between writing a SQL function (slow, interpreted) and writing a C function (fast, native) becomes clear and useful.

You are not implementing a custom index type, planner hooks, or background workers. Those are advanced features that real extensions sometimes use; they are out of scope here. Your effort goes into four things: getting the build system working, defining the type, writing the functions and operators, and demonstrating it all with a comprehensive SQL test script that doubles as the project's deliverable.

2 What the Final Product Looks Like

When your project is complete, a user can clone your repository, run `make install`, and then use the type from `psql` like this:

```
$ make
$ sudo make install
$ psql mydb

mydb=# CREATE EXTENSION semver;
CREATE EXTENSION

mydb=# SELECT '1.2.3'::semver;
 semver
--------
 1.2.3
(1 row)

mydb=# SELECT '1.2.3'::semver < '1.10.0'::semver;
 ?column?
----------
 t
(1 row)

mydb=# CREATE TABLE packages (
```

```
  name    text PRIMARY KEY,
  version semver NOT NULL
);
CREATE TABLE

mydb=# INSERT INTO packages VALUES
  ('postgres',    '16.4.0'),
  ('redis',       '7.2.5'),
  ('nginx',       '1.27.1'),
  ('node',        '20.10.0'),
  ('rust',        '1.81.0');
INSERT 0 5

mydb=# SELECT * FROM packages WHERE version >= '2.0.0' ORDER BY version DESC;
   name   | version
----------+---------
 postgres | 16.4.0
 node     | 20.10.0
 redis    | 7.2.5
(3 rows)

mydb=# SELECT major(version), minor(version), patch(version) FROM packages
       WHERE name = 'postgres';
 major | minor | patch
-------+-------+-------
    16 |     4 |     0
(1 row)

mydb=# SELECT max(version) FROM packages;
   max
---------
 20.10.0
(1 row)

mydb=# SELECT name, version, is_compatible(version, '1.0.0'::semver) FROM packages
    ;
   name   | version | is_compatible
----------+---------+---------------
 postgres | 16.4.0  | f
 redis    | 7.2.5   | f
 nginx    | 1.27.1  | t
 node     | 20.10.0 | f
 rust     | 1.81.0  | t
(5 rows)

mydb=# SELECT bump_minor('1.2.3'::semver);
 bump_minor
------------
 1.3.0
(1 row)
```

The point to notice is that PostgreSQL is using your code to do everything: parsing the literal `'1.2.3'` into a stored value, comparing two versions for the `<` operator (and noticing that `1.10.0` is greater than `1.2.3`, which a naive string comparison would get wrong), running the `MAX` aggregate over a column, calling the `is_compatible` function on each row, and printing the results. PostgreSQL's planner and executor have no special knowledge of `semver`; everything works because your extension correctly registered each function and operator with the right signatures.

3 Prerequisites and Environment Setup

This project requires a working PostgreSQL development environment. On Ubuntu/Debian:

```
sudo apt update
sudo apt install postgresql postgresql-server-dev-all build-essential
```

The crucial package is `postgresql-server-dev-all`, which installs the PostgreSQL header files and the `pg_config` tool that your `Makefile` will need. After installation, verify the toolchain works:

```
pg_config --includedir-server
pg_config --pkglibdir
```

Both commands must print valid directory paths. You should also create a database to test in (`createdb mydb`) and verify you can connect with `psql mydb`. Do all your development as a regular Linux user; only `make install` needs `sudo` (it copies your compiled `.so` file into PostgreSQL's library directory).

This project is C only. PostgreSQL's internal API is a C API and using C++ introduces compilation complications (name mangling, `extern "C"` wrapping, exception handling) that are not worth the effort for the small amount of code involved.

4 PostgreSQL Extension Mechanics

Before writing any code you need to understand five concepts. None of them is hard, but all of them are different from ordinary C programming, and ignoring them produces extensions that crash the entire PostgreSQL server.

4.1 The PGXS Build System

PostgreSQL ships with a Makefile fragment called PGXS that knows how to compile extension code against the right headers, link the right libraries, and install the result in the right place. Your `Makefile` is short:

```
MODULE_big = semver
OBJS       = semver.o

EXTENSION = semver
DATA      = semver--1.0.sql
REGRESS   = semver_test

PG_CONFIG = pg_config
PGXS := $(shell $(PG_CONFIG) --pgxs)
include $(PGXS)
```

Run `make` to compile, `sudo make install` to install, `make installcheck` to run the regression tests against an installed copy. PGXS handles every detail of finding the right include paths and the right output directory. You do not need to understand how PGXS works internally; you only need to fill in the four variables at the top.

4.2 Memory Contexts and palloc

PostgreSQL has its own memory allocator. You must use it instead of `malloc` and `free` for any memory whose lifetime is tied to the query being executed. The functions are `palloc(size)` and `pfree(ptr)`, and they look almost identical to their standard counterparts. The crucial difference is that PostgreSQL groups allocations into *memory contexts*. When a query finishes (or fails with an error), PostgreSQL destroys the per-query memory context, automatically freeing every `palloc` that was made inside it. You usually do not need to call `pfree` explicitly.

The rule for this project is: **always use palloc, never use malloc**. If you slip and use `malloc`, your extension will leak memory on every query; if you mix `malloc` and `pfree` (or vice versa), the server will crash.

4.3 The Datum Type

PostgreSQL passes all values between functions as `Datum`, a pointer-sized integer that can hold either a small value directly (an integer, a boolean) or a pointer to a larger value (a string, a struct). To get the actual value out of a `Datum` you use a conversion macro; to put one in you use the inverse macro. The macros are named after the C type they produce or consume:

```
int32 x = PG_GETARG_INT32(0);     // get 1st argument as int32
char *s = PG_GETARG_CSTRING(0);   // get 1st argument as null-terminated C string
MyType *p = (MyType *) PG_GETARG_POINTER(0); // get 1st arg as pointer to your
    type

PG_RETURN_INT32(42);              // return an int32
PG_RETURN_BOOL(true);             // return a boolean
PG_RETURN_CSTRING(s);             // return a null-terminated C string
PG_RETURN_POINTER(p);             // return a pointer to your type
```

You do not need to understand how the macros work internally. Treat them as the only legal way to read function arguments and return function results, and follow the pattern.

4.4 The Function Manager

Every C function you want to call from SQL must follow a fixed signature:

```
PG_FUNCTION_INFO_V1(my_function);

Datum my_function(PG_FUNCTION_ARGS) {
    /* read arguments with PG_GETARG_*, return with PG_RETURN_* */
}
```

The macro `PG_FUNCTION_INFO_V1(name)` declares your function to PostgreSQL using the modern ("version 1") calling convention. The function itself takes no explicit arguments; arguments come through the hidden `PG_FUNCTION_ARGS` macro. The function returns `Datum`; use the `PG_RETURN_*` macros to construct it. Every C function in your extension must follow exactly this signature.

4.5 The Magic Block

Every shared library that PostgreSQL loads as an extension must declare a "magic block" so that PostgreSQL can verify the library was compiled against the same version of PostgreSQL that is running. You include exactly one of these in exactly one source file:

```
#include "postgres.h"
#include "fmgr.h"

PG_MODULE_MAGIC;
```

Forget this and the server will refuse to load your extension with an error message about a missing magic block. Include it in two files and you will get a compilation error about a duplicate symbol. Include it in exactly one file.

5 The semver Type

The data your type will store is a struct of three 32-bit unsigned integers:

```
typedef struct {
    int32 major;
    int32 minor;
    int32 patch;
} Semver;
```

The text representation is the standard semver format: `MAJOR.MINOR.PATCH`, where each component is a non-negative decimal integer. `1.2.3`, `0.0.1`, `16.4.0`, and `20.10.0` are valid; `1.2`, `1.2.3.4`, `v1.2.3`, and `1.2.x` are not. To keep this project simple, you do not need to support semver pre-release labels (`1.0.0-rc.1`) or build metadata (`1.0.0+abc123`); both are part of the full semver specification but introduce comparison rules that are not worth the complexity here. Reject any input string containing a hyphen or a plus sign.

5.1 Comparison Rules

Two semvers are compared field by field, in order: `major` first, then `minor` if the majors are equal, then `patch` if both are equal. Comparison within each field is integer comparison, not string comparison; `1.10.0` is greater than `1.9.0`, even though as strings `"1.10.0"` comes before `"1.9.0"` lexicographically. This is the entire reason the type exists.

5.2 Compatibility Rule

Two versions are *compatible* (in the sense of the `is_compatible` function in the example session) if they share the same `major` number and the second one's `minor.patch` is less than or equal to the first one's. This is the same compatibility rule that npm uses for its caret (`^`) operator: `1.5.0` is compatible with `1.4.7` (it has everything 1.4.7 had, plus more), but `2.0.0` is not compatible with `1.9.0` (the major version bump signals breaking changes).

6 Phase 1 — Build System and Type Registration

The goal of Phase 1 is a working extension that can be installed and that supports `CREATE EXTENSION semver`, `SELECT '1.2.3'::semver`, and storing the type in a table column. No comparisons, no operators, no functions yet — just the type itself.

Build these components:

- The `Makefile` from Section 4.1, with `MODULE_big`, `OBJS`, `EXTENSION`, and `DATA` filled in correctly.
- A `semver.control` file containing the metadata PostgreSQL needs to recognize the extension:

```
comment = 'semantic version data type'
default_version = '1.0'
relocatable = true
module_pathname = '$libdir/semver'
```

- A `semver-1.0.sql` script that will be run by `CREATE EXTENSION`. At minimum it must declare the type as a "shell" (a placeholder), declare the input and output functions, and then complete the type definition:

```
CREATE TYPE semver;

CREATE FUNCTION semver_in(cstring)
  RETURNS semver
  AS 'MODULE_PATHNAME', 'semver_in'
  LANGUAGE C IMMUTABLE STRICT;

CREATE FUNCTION semver_out(semver)
  RETURNS cstring
  AS 'MODULE_PATHNAME', 'semver_out'
  LANGUAGE C IMMUTABLE STRICT;

CREATE TYPE semver (
  INPUT          = semver_in,
  OUTPUT         = semver_out,
  INTERNALLENGTH = 12,
  ALIGNMENT      = int4
);
```

- A C source file `semver.c` containing the magic block, the `Semver` struct, and two functions:
  - `semver_in` takes a C string, parses it into a `Semver` struct (use `sscanf` or write a small parser by hand), `palloc`s a struct of size 12 bytes, fills in the three fields, and returns it via `PG_RETURN_POINTER`. If the string does not match the format `INT.INT.INT` with no extra characters, call `ereport(ERROR, ...)` to abort with a clear message; PostgreSQL will turn this into a SQL error visible to the client.
  - `semver_out` takes a `Semver` pointer and produces a C string. Use `palloc` for the result buffer (32 bytes is more than enough), write the formatted string with `snprintf`, and return it via `PG_RETURN_CSTRING`.

At the end of Phase 1, this should work end to end:

```
$ make && sudo make install
$ psql mydb
mydb=# CREATE EXTENSION semver;
mydb=# SELECT '1.2.3'::semver;
 semver
--------
 1.2.3
mydb=# SELECT '1.2.x'::semver;
ERROR:  invalid input syntax for type semver: "1.2.x"
mydb=# CREATE TABLE t (v semver);
mydb=# INSERT INTO t VALUES ('5.6.7'), ('1.2.3');
mydb=# SELECT * FROM t;
   v
-------
 5.6.7
 1.2.3
```

7 Phase 2 — Functions and Operators

The goal of Phase 2 is to make the type useful: comparisons, operators, and a small collection of utility functions.

7.1 Comparison Functions

Add seven C functions to `semver.c`: `semver_lt`, `semver_le`, `semver_eq`, `semver_ne`, `semver_ge`, `semver_gt`, and `semver_cmp`. The first six return `bool`; the last returns `int32` that is negative, zero, or positive depending on the relative order of the two arguments. All seven internally call a single helper function, `semver_compare(a, b)`, which does the field-by-field integer comparison described in Section 5.1.

For each comparison function, register it in the SQL script with `CREATE FUNCTION` (using the same pattern as `semver_in`), then create the corresponding operator with `CREATE OPERATOR`:

```
CREATE FUNCTION semver_lt(semver, semver)
  RETURNS bool
  AS 'MODULE_PATHNAME', 'semver_lt'
  LANGUAGE C IMMUTABLE STRICT;

CREATE OPERATOR < (
  LEFTARG    = semver,
  RIGHTARG   = semver,
  PROCEDURE  = semver_lt,
  COMMUTATOR = >,
  NEGATOR    = >=
);
```

The `COMMUTATOR` and `NEGATOR` clauses are hints to the query planner: if it knows that `a < b` can be rewritten as `b > a` (commutator) or as `NOT (a >= b)` (negator), it can sometimes simplify queries. Repeat for all six operators (`<`, `<=`, `=`, `<>`, `>=`, `>`).

After this is done, `ORDER BY` and `WHERE version > '1.0.0'` both work in `psql`, because PostgreSQL's planner uses the operators you registered.

7.2 Utility Functions

Add five more C functions and register them in the SQL script:

- `major(semver) returns int` — return the `major` field of the input.
- `minor(semver) returns int` — return the `minor` field.
- `patch(semver) returns int` — return the `patch` field.
- `bump_minor(semver) returns semver` — return a new semver with the same `major`, the `minor` incremented by 1, and `patch` reset to 0. Allocate the result with `palloc`.

- `is_compatible(semver, semver) returns bool` — return true if the first argument's `major` equals the second argument's `major` and the second argument's (`minor`, `patch`) is lexicographically $\leq$ the first's. This implements the npm caret rule from Section 5.2.

These functions exercise everything you have learned: argument extraction, palloc-based result construction, struct field access, and SQL registration.

7.3 Verifying with EXPLAIN

Once the operators are registered, run a query in `psql` prefixed with `EXPLAIN`:

```
mydb=# EXPLAIN SELECT * FROM packages WHERE version >= '2.0.0';
```

The plan should show a sequential scan with a filter expression that uses your `semver_ge` function. This confirms that PostgreSQL's planner found and is using your operator. There is no index on the column (you have not registered an operator class for B-tree, which is intentionally outside the scope of this project), so the scan is sequential. That is fine and expected.

8 Phase 3 — Aggregate, Tests, and Demo

The goal of Phase 3 is to add a custom aggregate, write a comprehensive regression test, and produce a demo SQL file that showcases everything.

8.1 The MAX Aggregate

PostgreSQL's built-in `MAX` function is itself an aggregate that takes a state-transition function and a comparison. To make `MAX(semver_column)` work, you register a state-transition function and then declare an aggregate.

The state-transition function takes the current running maximum and the next value, and returns whichever is larger. In C:

```
PG_FUNCTION_INFO_V1(semver_larger);
Datum semver_larger(PG_FUNCTION_ARGS) {
    Semver *a = (Semver *) PG_GETARG_POINTER(0);
    Semver *b = (Semver *) PG_GETARG_POINTER(1);
    PG_RETURN_POINTER(semver_compare(a, b) >= 0 ? a : b);
}
```

In SQL:

```
CREATE FUNCTION semver_larger(semver, semver)
  RETURNS semver
  AS 'MODULE_PATHNAME', 'semver_larger'
  LANGUAGE C IMMUTABLE STRICT;

CREATE AGGREGATE max(semver) (
  SFUNC    = semver_larger,
  STYPE    = semver
);
```

That is the entire aggregate. The first row's value becomes the initial state. Each subsequent row is fed through the state function. The final state is the result. PostgreSQL handles the loop, the parallelism, and the call to your function.

Repeat the pattern with `semver_smaller` and `CREATE AGGREGATE min(semver)`.

8.2 Regression Tests

PostgreSQL ships with a test framework called `pg_regress` that PGXS integrates with automatically. To use it, create two files:

```
sql/semver_test.sql       -- a SQL script of test queries
expected/semver_test.out  -- the exact expected output of those queries
```

`make installcheck` runs your script against an installed extension and compares the output character-for-character against the expected file. Any difference is a failure. Cover at least:

- Basic input and output (`SELECT '1.2.3'::semver`).
- Input rejection for malformed strings (the test should expect the SQL error message your `ereport` produces).
- All six comparison operators with cases that exercise the field-by-field comparison (especially `1.10.0` vs. `1.9.0` to verify it is not a string compare).
- All five utility functions.
- `ORDER BY` on a column of `semver`.
- `MAX` and `MIN` aggregates.
- A query that combines several features (e.g., the `is_compatible` demo from the example session).

The expected output files take some patience to construct. The recommended workflow is to run your script in `psql`, copy the output, and clean it up by hand. Once `make installcheck` passes, leave the expected files alone and treat any future test failure as a real bug to investigate.

8.3 Demo Script

In addition to the regression tests, write a single SQL file `demo.sql` that loads a realistic dataset (a table of around 20 software packages with versions) and runs the queries from the example session in Section 2. The intent is for a grader to be able to type `psql mydb -f demo.sql` and see your entire feature set in action in under one screen of output.

9 Deliverables and Submission

9.1 What to Submit

A single zip archive containing:

- The complete source tree of your extension: `semver.c`, `semver.control`, `semver-1.0.sql`, the `Makefile`, the `sql/` and `expected/` directories, and `demo.sql`.
- A `README.md` with build instructions, the list of system packages required (including the PostgreSQL development headers), the list of supported types, functions, operators, and aggregates, and any known limitations.
- A `design.pdf` document of at most 8 pages explaining: the structure of the extension (which file does what), the role of the magic block and PGXS, the in-memory representation of the type, the comparison helper, the SQL registration pattern, the regression test strategy, and a brief discussion of one design decision you made (for example, why you used a struct of three int32 fields rather than packing them into a single int64).

9.2 Archive Naming

`Group[Number]_Project02_PGExtension.zip`, for example `Group17_Project02_PGExtension.zip`.

9.3 Demonstration

Each group will present a live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a brief walkthrough of the source layout using the design document. Second, a live build and install of the extension on a clean database, followed by execution of the regression tests and the demo script, with the instructor free to type new queries on the spot. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation: what the magic block does, what the difference between `palloc` and `malloc` is, what `PG_FUNCTION_INFO_V1` accomplishes, or how the SQL planner finds your operators. All group members must be present. Missing the demonstration results in a zero for Phase 3.

10 Bonus Opportunities

Each of the following is independent and may be attempted in any combination. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points total. To claim bonus credit, your design document must include a dedicated section explaining which bonus items you attempted and how you verified them.

10.1 B-tree Operator Class for Indexing (up to 10%)

Register a B-tree operator class for `semver` so that `CREATE INDEX` works on `semver` columns. This requires the seven comparison functions (which you already have for the base project) to be assigned to specific "strategy numbers" inside an `OPERATOR CLASS` declaration. The PostgreSQL documentation chapter on "Interfacing Extensions to Indexes" describes the strategy numbers for B-tree (1 through 5 for `<`, `<=`, `=`, `>=`, `>`) and the support function (`semver_cmp`). Your demo must include an `EXPLAIN` showing that an indexed range query uses an `Index Scan` rather than a `Seq Scan`.

10.2 Pre-release Labels (up to 5%)

Extend the type to support semver pre-release labels (`1.0.0-alpha`, `1.0.0-rc.1`). The comparison rules in section 11 of the semver 2.0.0 specification become more complex: a version with a pre-release label is *less* than the same version without one (`1.0.0-rc.1` < `1.0.0`), and pre-release labels themselves are compared component by component with mixed numeric and lexical rules. Implement the rules correctly and add regression tests that exercise every case in the specification. Your design document must explain the comparison algorithm in detail.

10.3 Custom Cast to and from text (up to 5%)

Register an explicit cast between `text` and `semver` (`CREATE CAST (text AS semver)` and `CREATE CAST (semver AS text)`) so that users can write `some_text_column::semver` naturally. The cast functions are thin wrappers around your existing input and output functions, but the registration is its own learning exercise.

11 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing semver or version-comparison library (`libsemver`, `semver4j`, the `semver` npm package, etc.). All parsing and comparison logic must be written from scratch.
- Use of `malloc`, `calloc`, `realloc`, or `free` for memory whose lifetime spans a PostgreSQL function call. Use `palloc` and (if necessary) `pfree`.
- Use of any language other than C. C++ is not permitted because it complicates the build and introduces ABI issues with PostgreSQL's C API.
- Implementing the type as a SQL function (`LANGUAGE plpgsql`) instead of a C function. The whole point of the project is the C extension API.
- Skipping the regression tests. `make installcheck` must pass before submission.
- Submitting an extension that crashes the PostgreSQL server when used. A crash during grading is a failing grade for Phase 3 regardless of how much else works.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of an extension of this size; histories with three commits totaling thousands of lines will be scrutinized for academic integrity.

12 Required Reading

The first item is required reading before you start. The remaining items are reference material to consult during implementation; they are PostgreSQL's official documentation, which is generally good but is organized as a reference rather than a tutorial.

1. Percona Blog, "Writing PostgreSQL Extensions is Fun – C Language," 2018. The clearest introductory walkthrough of building a simple extension end to end. Read it before Phase 1; the structure of the example matches what you will be doing.
2. PostgreSQL official documentation, chapter "Extending SQL," specifically the sections on user-defined types and user-defined functions in C. The reference for everything in Sections 4 and 5 of this manual.
3. PostgreSQL official documentation, chapter "Server Programming Interface," the section on memory

management with `palloc` and memory contexts. Skim it; you will not need most of it, but the basic rules are essential.

4. PostgreSQL official documentation, chapter "Interfacing Extensions to Indexes." Required reading only if you attempt the B-tree operator class bonus.
5. The source code of `contrib/citext` in the PostgreSQL source tree (a small extension that adds a case-insensitive text type). It is the closest in spirit and complexity to what you will build, and reading it is the single most useful thing you can do when stuck.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM